

Software Requirements Specification For NextStay

A Distributed Peer-to-Peer Accommodation Marketplace

Version 1.0

<Annotated Version>

April 15, 2026

Prepared by

Fatima Swelem 20230401

Shaymaa Ghanim 20230292

Sondos Mohamed 20230268

Dana Mohamed 20230190

Habiba Khaled 20230171

Mohamed Harby 20230462

Table of Contents

1.0 Introduction

1.1 Purpose

1.2 Project Scope

1.3 Glossary and Abbreviation

1.4 System Stakeholders

1.5 References

Requirements Specification

2.0 User Requirements Specification (URS)

2.1 Functional User Requirements

2.2 Non-Functional User Requirements

3.0 System Requirements Specification

3.1 Functional System Requirements

3.1.1 Module 1: Identity & Access Management Service

3.1.2 Module 2: Listing & Catalog Service

3.1.3 Module 3: Booking & Reservation Engine

3.1.4 Module 4: Review & Feedback Service

3.1.5 Module 5: Customer Service & Support

3.1.6 Administrative Functions

3.2 Functional Requirements' Priorities (using MoSCoW Scheme)

3.3 Non-Functional System Requirements

3.3.1 Technical & Architectural Constraints 3.3.2 Security

3.3.3 Data & Persistence 3.3.4 Performance

3.3.5 Reliability & Availability 3.3.6 Maintainability

4.0 Design & Implementation Constraints

5.0 System Evolution

6.0 Requirements Discovery Approaches

7.0 Requirements Validation Techniques

8.0 System Design and Models

8.1 Use-Case Diagram

8.1.1 Detailed Use-Cases Description

8.2 Activity Diagrams

8.3 Class Diagram

8.4 Sequence Diagrams

8.5 Entity Relationship Diagram (ERD)

9.0 System Architecture Overview

9.1 Service Decomposition

9.2 Cross-Cutting Concerns (AOP)

9.3 Docker Compose Structure

9.4 System Architecture

9.5 List of User Interfaces

9.6 Design Pattern

9.7 The Strategy/s used to implement the use cases

1.0 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete set of functional and non-functional requirements for NextStay?, a distributed, peer-to-peer accommodation marketplace. The platform connects property owners (Hosts) with travelers (Guests) through a cloud-native, microservices architecture built on Spring Boot and Spring Cloud.

This document serves as:

- A single source of truth for developers, designers, and stakeholders to align on feature implementation.
- A quality assurance benchmark for validating performance, security, and usability.
- A compliance guide ensuring adherence to Software Engineering-2 project requirements including microservices, AOP, Dockerization, and RBAC.

1.2 Project Scope

NextStay? is scoped as a focused accommodation marketplace inspired by Airbnb, but intentionally limited to the core rental lifecycle. The system encompasses:

- User identity management (registration, authentication, role-based access).
- Property listing creation, search, and management by Hosts.
- Reservation creation, status tracking, and double-booking prevention for Guests.
- Post-stay review and rating submission and aggregation.
- Platform administration: host verification and listing moderation by Admins.

Out of scope:

payment processing, social features, map integrations, and mobile- native applications.

In-platform real-time chat between Guests and Hosts is also out of scope; direct communication is facilitated through the Customer Service ticketing module.

3. Glossary and Abbreviation

Term / Abbreviation	Definition
JWT	JSON Web Token, a compact, self-contained token used for secure stateless authentication.
RBAC	Role-Based Access Control, restricts system access to authorized users based on their assigned role.
AOP	Aspect-Oriented Programming, a paradigm for centralizing cross-cutting concerns such as logging and security.
REST API	Representational State Transfer Application Programming Interface, architectural style for distributed hypermedia systems.
Microservice	An independently deployable service responsible for a single bounded business domain.
Spring Cloud	A suite of tools for building cloud-native Spring Boot microservices (Gateway, Eureka, Config Server).
Eureka	Netflix Eureka, a service registry used for dynamic service discovery within the microservices ecosystem.
Docker	A containerization platform used to package services and their dependencies for consistent deployment.
OCL	Object Constraint Language, formal language for expressing constraints on UML models.
User	A placeholder for the use of both Host and Guest Roles
Guest	An authenticated user who searches for properties, makes reservations, and writes reviews.
Host	A user who creates and manages property listings and approves or declines booking requests.
Users' Admin	Manages the user-facing platform: approves/rejects listings, flags users (guests/hosts), oversees host reviews/ratings, initiates refunds, and resolves action-needed tickets
Employees' Admin	Manages internal corporate employees (support agents): creates/deactivates accounts, assigns tickets to agents, and monitors agent performance metrics.
ERD	Entity Relationship Diagram, a structural view of the database schema.
Customer Service (CS) Service	A dedicated microservice that manages user-submitted support tickets, agent responses, and ticket lifecycle (Open → In Progress → Resolved → Closed).
Support Ticket	A structured request submitted by a Guest or Host to the platform's support team, containing a subject, description, and current status.
Support Agent	A system role responsible for reviewing, responding to, and resolving support tickets.

4. System Stakeholders

Role	Responsibilities
Guest	Search for properties, book stays, submit reviews, manage personal reservations.
Host	Create and manage listings, set pricing and availability, respond to reviews, accept/decline bookings.
Employees' Admin	Manages internal corporate employees (support agents): creates/deactivates accounts, assigns tickets to agents, and monitors agent performance metrics.
Users' Admin	Manages the user-facing platform: approves/rejects listings, flags users (guests/hosts), oversees host reviews/ratings, initiates refunds, and resolves actionneeded tickets
DevOps Engineers	Maintain Docker environment, CI/CD pipeline [continuous integration (CI) and continuous delivery/deployment (CD)], service health, and database integrity.
Database Administrator	Design and maintain isolated microservice databases; ensure data integrity and backup.
Support Agent	Reviews submitted support tickets, responds to users, escalates issues, and resolves or closes tickets.

1.5 References

- *Airbnb platform: <https://www.airbnb.com>*
- *Spring Boot Documentation: <https://spring.io/projects/spring-boot>*
- *Spring Cloud Documentation: <https://spring.io/projects/spring-cloud>*
- *SE-2 Project Guidelines — TA-issued requirements document (April 2026).*
- *CS251 Introduction to Software Engineering — Lecture Notes, Spring 2025.*

Requirements Specification

2.0 User Requirements Specification (URS)

2.1 Functional User Requirements

ID	Requirement Description
URS-01	Users must be able to register a new account using their full name, email and password.
URS-02	Users must be able to log in securely using their registered emails/password and receive a session token.
URS-03	Users/Employees must be able to log out and terminate their session.
URS-04	Users must be able to update their personal profile information.
URS-05	Guests must be able to search for properties by location and date range and host
URS-06	Guests must be able to view Hosts' overall rating (optional)
URS-07	Guests must be able to view detailed property information, photos, and amenities.
URS-08	Guests must be able to request a booking for specific check-in and check-out dates.
URS-09	Guests must be able to view the status of their reservations (Pending, Confirmed, Completed, Cancelled, Rejected).
URS-10	Guests must be able to cancel a reservation according to the cancellation policy.
URS-11	Guests must be able to submit a rating and written review after a completed stay.
URS-12	Guests must be able to delete their own written reviews
URS-13	Hosts must be able to create new property listings with descriptions, amenities, and pricing.
URS-14	Hosts must be able to manage an availability calendar to block or open dates.
URS-15	Hosts must be able to accept or decline pending booking requests.
URS-16	Hosts must be able to respond to guest reviews on their properties. (optional)
URS-17	Hosts must be able to edit or remove their property listings.
URS-18	Users' Admins must be able to manage (suspend if needed) all users' accounts on the platform.
URS-19	Users' Admins must be able to approve/flag pending listings
URS-20	Users' Admin must be able to view and resolve action needed tickets, filtered by type (refund or deactivation)
URS-21	Users' Admin must be able to oversee hosts' reviews and rating (optional)
URS-22	Employees' Admin must be able to view Employees' performance dashboard (tickets resolved per agent, overall resolved vs. pending).
URS-23	Employees' Admin must be able to create/deactivate employees
URS-24	Employees' Admin must be able to Assign tickets to specific employees (SA)
URS-25	The system shall notify the Guest if their selected dates are unavailable and prompt them to choose a different date range.

URS-26	The system must support distinct roles: Guest, Host, Support Agent, Users' Admin and Employees' Admin, with separate access permissions.
URS-27	Users must be able to submit a support ticket describing their issue
URS-28	Users must be able to view all their submitted tickets and track their status (Open, In Progress, Resolved, Closed).
URS-29	Users must receive a notification when a Support Agent responds to or resolves their ticket. (optional)
URS-30	Users must receive notification when a refund is processed (optional)
URS-31	Support Agents must be able to view assigned tickets and mark tickets as resolved/action needed (specify refund/deactivation)
URS-32	Support Agents must be able to attach a message to a ticket for ticket initiator to read
URS-33	Support Agents must be able to flag (hide/delete) a guest's review

2.2 Non-Functional User Requirements

ID	Requirement Description
URSN-01	The system shall respond to user search queries within 3 seconds under normal load.
URSN-02	The platform shall be available 99.5% of the time, ensuring reliable access for users and hosts.
URSN-03	User passwords and sensitive data shall be securely stored using industry-standard encryption.
URSN-04	The system shall provide meaningful error messages when a user performs an invalid action.
URSN-05	The platform shall support concurrent use by multiple users without performance degradation.
URSN-06	The user interface shall be intuitive such that a new user can complete a booking in fewer than five steps.

3.0 System Requirements Specification

The system is divided into five functional modules, each implemented as an independent microservice. The first four modules satisfy the minimum project requirement of four modules; the fifth module: Customer Service, extends the platform with user support capabilities:

3.1 Functional System Requirements

3.1.1 Module 1: Identity & Access Management Service

FR-01: User Registration

Function	Allow new users to create an account on the NextStay? platform.
Inputs	Full name, email address, password, and selected role (Guest or Host).
Outputs	Confirmation message; user record created; role assigned in the system.
Information Used	User credentials stored in the Identity Service database.
Action	Validate email uniqueness, hash password, persist user, assign role, and return success.
Pre-conditions	The email address must not already be registered in the system.
Post-conditions	A user account with a unique ID and assigned role exists in the database.
Side Effects	Role assignment record created; JWT readiness established for subsequent login.

FR-02: User Authentication (Login)

Function	Authenticate a registered user and issue a JWT for API access.
Inputs	Registered email address and password.
Outputs	A signed JWT token; user dashboard access granted.
Information Used	Hashed credentials from the Identity Service database.
Action	Compare hashed input with stored credential; generate and return JWT on success.
Pre-conditions	User must have a registered account.
Post-conditions	User session established; JWT issued and valid for a configured time window.
Side Effects	Login timestamp recorded; failed attempt counter updated if applicable.

FR-03: Role Management & API Authorization

Function	Assign roles to users and enforce RBAC across all API endpoints.
Inputs	User role assigned at registration; JWT claims present in each API request header.
Outputs	Permitted access to role-specific endpoints; 403 Forbidden response for unauthorized attempts.
Information Used	JWT claims, role definitions stored in the Identity Service.
Action	Spring Security validates JWT; AOP intercepts and checks role before routing request.
Pre-conditions	User is authenticated; JWT is valid and unexpired.
Post-conditions	Request either proceeds to the target service or is rejected with an error.
Side Effects	AOP security audit log entry created for every authorized and unauthorized access attempt.

FR-04: User Profile Management

Function	Allow users to view and update their personal profile information.
Inputs	Updated name, phone number, profile photo, or bio from authenticated user.
Outputs	Updated profile record; confirmation of changes.
Information Used	Existing profile data in the Identity Service database.
Action	Validate input fields; update the user record; return updated profile.
Pre-conditions	User must be authenticated.
Post-conditions	Profile reflects the latest submitted information.
Side Effects	AOP logging records the update action with a timestamp.

3.1.2 Module 2: Listing & Catalog Service

FR-05: Property Listing Creation

Function	Allow verified Hosts to create new property listings.
Inputs	Property name, description, location (city), amenities list, photos, nightly price, and maximum guests.
Outputs	New listing record created; listing marked as Inactive verification.
Information Used	Host identity from JWT; listing data stored in Listing Service database.
Action	Validate required fields; persist listing; notify Admin for approval; return listing ID.
Pre-conditions	User must be authenticated with the Host role.
Post-conditions	Listing exists in the database with status Inactive; listing is not publicly visible until approved.

Side Effects	Admin notification triggered; AOP logs the creation event.
---------------------	--

FR-06: Property Search

Function	Allow Guests to search for available properties by location and date range.
Inputs	Location (city string), check-in date, check-out date, and optional filters (max guests, max price).
Outputs	A paginated list of available listings matching the search criteria.
Information Used	Listing database; Booking Service queried for availability cross-check.
Action	Filter active listings by location; call Booking Service to exclude dates with confirmed reservations; return results.
Pre-conditions	At least one active listing must exist.
Post-conditions	Only available, Admin-approved listings are returned.
Side Effects	None.

FR-07: Availability Calendar Management

Function	Allow Hosts to manage the availability calendar for their properties.
Inputs	Selected date range and action (block or open dates); Host JWT.
Outputs	Updated availability calendar for the property.
Information Used	Existing availability records for the property in the Listing Service database.
Action	Validate dates; update availability slots; prevent editing confirmed booking dates.
Pre-conditions	Host must own the listing; dates must not overlap with confirmed reservations.
Post-conditions	Calendar reflects the updated availability.
Side Effects	Blocked dates automatically excluded from search results.

FR-08: Listing Verification by Users' Admin

Function	Allow Users' Admins to review and approve or flag property listings for quality control.
Inputs	Listing ID and Users' Admin decision (Approved / Flagged) with optional reason.
Outputs	Listing status updated; Host notified of decision.
Information Used	Listing records in the Listing Service database; Admin JWT.

Action	Users' Admin reviews listing details; updates listing status; triggers notification to Host.
Pre-conditions	Listing must exist with status Inactive; Users' Admin must be authenticated.
Post-conditions	Approved listings become publicly searchable; flagged listings are hidden.
Side Effects	AOP audit log records Users' Admin decision with timestamp and reason.

3.1.3 Module 3: Booking & Reservation Engine

FR-09: Reservation Creation

Function	Allow an authenticated Guest to request a booking for a specific property and date range.
Inputs	Listing ID, check-in date, check-out date, number of guests; Guest JWT.
Outputs	New reservation record with status Pending; Host notified.
Information Used	Listing data (from Listing Service); existing reservations in Booking Service database.
Action	Verify listing is active; check date availability; prevent double-booking; create reservation; notify Host.
Pre-conditions	Guest must be authenticated; listing must be active and available for the requested dates.
Post-conditions	Reservation exists with status Pending; requested dates are soft-locked pending Host confirmation.
Side Effects	Date availability updated; AOP logs reservation creation event.

FR-10: Double-Booking Prevention

Function	Ensure that no two reservations overlap for the same property.
Inputs	Requested check-in and check-out dates; target listing ID.
Outputs	Booking proceeds if dates are free; 409 Conflict error if overlap detected.
Information Used	All existing Confirmed and Pending reservations for the target listing.
Action	Query existing reservations; apply date-range overlap logic; reject booking if conflict exists.
Pre-conditions	Booking request is in progress.
Post-conditions	Only one reservation holds the requested dates per property.
Side Effects	Distributed lock may be used to prevent race conditions under concurrent booking attempts.

FR-11: Booking Status Lifecycle Management

Function	Track and transition reservation statuses throughout the booking lifecycle.
Inputs	Reservation ID; triggering action (Host approval, Guest cancellation, system check-out detection).
Outputs	Updated reservation status; notifications sent to affected parties.
Information Used	Reservation record; current date (for auto-completion); role of requesting
Action	Validate the status transition; update the record; trigger notification; log transition.
Pre-conditions	Reservation must exist and be in a valid state for the requested transition.
Post-conditions	Reservation reflects the new status. Valid transitions: Pending → Confirmed → Completed; Pending → Rejected (Host declines); Confirmed → Cancelled (Guest cancels).
Side Effects	Completed reservations unlock the ability for Guests to submit reviews.

FR-12: Automated Stay Completion

Function	Automatically mark reservations as Completed once the check-out date has passed.
Inputs	System clock (scheduler trigger); all Confirmed reservations with past check-out dates.
Outputs	Reservation status updated to Completed.
Information Used	Booking Service database; current system date.
Action	A scheduled job runs daily; identifies overdue Confirmed reservations; updates their status to Completed.
Pre-conditions	Reservation must be in Confirmed status; check-out date must be in the past.
Post-conditions	Reservation is marked Completed; Guest becomes eligible to submit a review.
Side Effects	AOP logs the automated transition.

3.1.4 Module 4: Review & Feedback Service

FR-13: Guest Review Submission

Function	Allow a Guest to submit a rating and written review for a completed stay.
Inputs	Reservation ID (must be Completed); star rating (1–5); written review text; Guest JWT.
Outputs	Review record created and associated with the listing and the host.
Information Used	Booking Service queried to verify reservation is Completed; Review Service database.
Action	Verify reservation is completed and belongs to the requesting Guest; persist review; trigger rating aggregation.
Pre-conditions	Reservation must have status Completed; Guest must not have previously reviewed this reservation.
Post-conditions	Review is visible on the property listing; average rating for the property is recalculated.
Side Effects	Host notified of new review.

FR-14: Host Review Response

Function	Allow a Host to publicly respond to a Guest review on their property.
Inputs	Review ID; response text; Host JWT.
Outputs	Host response attached to the review and publicly visible.
Information Used	Review record; Host identity verification.
Action	Verify the Host owns the listing associated with the review; persist response.
Pre-conditions	Review must exist; Host must own the property that was reviewed.
Post-conditions	Response is visible alongside the original review.
Side Effects	None.

FR-15: Rating Aggregation

Function	Calculate and maintain the average rating for each property and host.
Inputs	New review submission event.
Outputs	Updated average rating displayed on the listing.
Information Used	All approved reviews for the target listing.
Action	Recalculate average on new review event; update listing rating field.
Pre-conditions	At least one review must exist for the property.
Post-conditions	Listing displays the accurate, up-to-date average rating.
Side Effects	None.

3.1.5 Module 5: Customer Service & Support

FR-16: Support Ticket Submission

Function	Allow authenticated Guests or Hosts to submit a support ticket to the platform's support team.
Inputs	Subject, description; JWT of submitting user.
Outputs	New ticket record created with status Open; ticket ID returned; user notified of submission.
Information Used	User identity from JWT; ticket data stored in Customer Service DB.
Action	Validate input fields; persist ticket with status Open; assign auto-generated ticket number; notify Support Agents.
Pre-conditions	User must be authenticated as Guest or Host.
Post-conditions	Ticket exists in the database with status Open and is visible to Support Agents.
Side Effects	AOP logs ticket creation event; Support Agent dashboard updated.

FR-17: Support Ticket Response by Agent

Function	Allow a Support Agent to read a ticket and post a response to the submitting user.
Inputs	Ticket ID; response message text; Agent JWT.
Outputs	Response appended to ticket thread; ticket status updated to In Progress; user notified.
Information Used	Ticket record in CS DB; Agent identity from JWT.
Action	Validate Agent role; handle ticket; append response message; update status to In Progress; trigger notification to ticket owner.
Pre-conditions	Ticket must exist with status Open or In Progress; requesting user must hold Agent role.
Post-conditions	Ticket contains Agent response; status is In Progress; user is notified.
Side Effects	AOP security audit log records Agent action.

FR-18: Ticket Status Lifecycle Management

Function	Track and transition support ticket statuses through their lifecycle.
Inputs	Ticket ID; new status (In Progress / Resolved / Closed); Agent or User JWT.
Outputs	Updated ticket status; notification sent to ticket owner.
Information Used	Ticket record; role of requesting user.
Action	Validate that the requesting role is permitted to perform the transition; update ticket status; log transition.
Pre-conditions	Ticket must exist; transition must be valid (Open → In Progress → Resolved → Closed).
Post-conditions	Ticket reflects the new status.
Side Effects	Resolved status notifies user and prompts optional satisfaction feedback.

FR-19: User Ticket History View

Function	Allow authenticated users to view all their past and current support tickets.
Inputs	JWT of authenticated user.
Outputs	List of the user's tickets with status, subject, and last-updated timestamp.
Information Used	CS DB filtered by userId.
Action	Query tickets by userId from JWT; return paginated results.
Pre-conditions	User must be authenticated.
Post-conditions	User sees their complete ticket history.
Side Effects	None

FR-20: Support Agent Ticket Dashboard

Function	Allow Support Agents to view and manage assigned support tickets.
Inputs	Filter criteria (status, date range); Agent JWT.
Outputs	Filtered list of tickets with submitter info, status, and timestamps.
Information Used	All ticket records in CS DB.
Action	Retrieve tickets matching filter; enable status transitions.
Pre-conditions	Requesting user must hold Support Agent role.
Post-conditions	Support Agent sees current ticket state; can take action on any ticket except (refunds/deactivation)
Side Effects	AOP logs Support Agent access to ticket data.

FR-21: SA Account Management by Employees' Admin

Function	Allow Employees' Admin to create and deactivate Support Agent accounts.
Inputs	Full name, email, password (for creation); SA account ID and deactivation reason (for deactivation); Employees' Admin JWT.
Outputs	New SA account created and active, or existing SA account deactivated and access revoked.
Information Used	User records in the Identity Service database; Employees' Admin JWT.
Action	Validate input fields; persist new SA user record with role Support Agent or update existing SA account status to Deactivated; return confirmation.
Pre-conditions	Employees' Admin must be authenticated. For deactivation, the target account must exist and hold the Support Agent role.
Post-conditions	New SA account is active and can log in, or deactivated SA loses all system access immediately.
Side Effects	AOP audit log records the Admin action with timestamp.

FR-22: Ticket Assignment by Employees' Admin

Function	Allow Employees' Admin to assign open or action-needed tickets to specific Support Agent accounts.
Inputs	Ticket ID, target SA account ID; Employees' Admin JWT.
Outputs	Ticket updated with assigned agent; SA notified of assignment.
Information Used	Ticket records in CS DB; SA user records in Identity Service; Employees' Admin JWT.
Action	Validate that ticket exists and is in Open or Action Needed status; validate target account holds SA role; update ticket with assigned AgentId; notify SA.
Pre-conditions	Employees' Admin must be authenticated; ticket must exist in Open or Action Needed status; target user must hold Support Agent role.
Post-conditions	Ticket reflects assigned SA; SA is notified and ticket appears in their dashboard.
Side Effects	AOP logs assignment action with timestamp.

FR-23: Employee Performance Dashboard

Function	Allow the Employees' Admin to view a performance dashboard showing ticket resolution metrics for each Support Agent.
Inputs	Employees' Admin JWT; optional filter by agent ID.
Outputs	Per-agent statistics: total tickets assigned, resolved count, pending count; platform-wide overall totals.
Information Used	All ticket records in CS DB filtered by assignedAgentId; aggregated counts per status.
Action	Query all tickets assigned to each agent; group and count by status; return per-agent map and overall summary.
Pre-conditions	Employees' Admin must be authenticated.
Post-conditions	Admin sees an up-to-date performance view per agent and platform totals.
Side Effects	AOP logs admin access to performance data with timestamp.

FR-24: Refund Initiation by Users' Admin (optional)

Function	Allow Users' Admin to mark a refund-flagged ticket as processed, after the refund action has been carried out externally.
Inputs	Ticket ID flagged with actionType = refund; Users' Admin JWT.
Outputs	Ticket status updated to Resolved; action-needed flag cleared.
Information Used	Ticket record in CS DB with actionNeeded = true and actionType = refund.
Action	Validate ticket exists with actionType refund; update ticket status to Resolved; clear actionNeeded flag; log action.
Pre-conditions	Users' Admin must be authenticated; ticket must exist with actionNeeded = true and actionType = refund.
Post-conditions	Ticket is resolved; no longer appears in the action-needed queue.
Side Effects	AOP audit log records Admin action with timestamp and ticket ID.

FR-25: Action-Needed Ticket Resolution by Users' Admin

Function	Allow Users' Admin to view, filter, and resolve tickets flagged as Action Needed by Support Agents.
Inputs	Filter criteria (action Type: Refund or Deactivation); Users' Admin JWT.
Outputs	Filtered list of Action Needed tickets; resolved ticket status after Admin action.
Information Used	All ticket records in CS DB with status Action Needed.
Action	Retrieve tickets filtered by status Action Needed and optional action Type; display to Admin; Admin selects a ticket and performs the required action (refund via FR-24, or user deactivation); update ticket status to Resolved.
Pre-conditions	Users' Admin must be authenticated; at least one ticket must exist with status Action Needed.
Post-conditions	Tickets are resolved; corresponding action (refund or user deactivation) is completed.
Side Effects	AOP audit log records all Admin actions with timestamp.

FR-26: Guest Review Deletion

Function	Allow an authenticated Guest to delete a review they previously submitted.
Inputs	Review ID; Guest JWT.
Outputs	Review record deleted; listing average rating recalculated.
Information Used	Review records in Review Service DB; Guest identity from JWT.
Action	Validate that review exists and belongs to the requesting Guest; delete review record; trigger rating recalculation for the associated listing.
Pre-conditions	Guests must be authenticated; review must exist and be owned by the requesting Guest.
Post-conditions	Review is permanently removed from the listing; listing average rating is updated.
Side Effects	AOP logs the deletion event with timestamp.

FR-27: Review Flagging by Support Agent

Function	Allow a Support Agent to flag a guest review as inappropriate for Users' Admin review.
Inputs	Review ID; flag reason; SA JWT.
Outputs	Review marked as Flagged; Users' Admin notified for moderation.
Information Used	Review records in Review Service DB; SA identity from JWT.
Action	Validate SA role; retrieve review; update review status to Flagged; attach flag reason; notify Users' Admin.
Pre-conditions	Requesting users must hold Support Agent role; review must exist and not already be flagged.
Post-conditions	Review is marked Flagged and surfaced to Users' Admin for a moderation decision.
Side Effects	AOP logs the flagging action with SA ID, review ID, and timestamp.

FR-28: Host Rating Visibility for Guest

Function	Allow an authenticated Guest to view a host's overall aggregated rating.
Inputs	Host user ID; Guest JWT.
Outputs	Host's aggregated average rating calculated across all their listings.
Information Used	Review records in Review Service DB associated with the host's listings.
Action	Retrieve all approved reviews across all active listings owned by the specified host; compute and return the aggregated average rating.
Pre-conditions	Guests must be authenticated; the specified host must have at least one reviewed listing.
Post-conditions	No system state changed; Guest views the host's overall rating.
Side Effects	AOP logs assignment action with timestamp.

3.2 Functional Requirements' Priorities (using MoSCoW Scheme)

Priority	Requirement IDs	Justification
Must Have	FR-01, FR-02, FR-03, FR-05, FR-06, FR-09, FR-10, FR-11	Core registration, authentication, listing, search, and booking — without these the system cannot function.
Should Have	FR-04, FR-07, FR-08, FR-12, FR-13, FR-15, FR-16, FR-17, FR-18, FR-19, FR-20, FR-21, FR-22, FR-24, FR-25	Profile management, calendar, verification, autocompletion, reviews, rating aggregation, and full customer service functionality, essential for a complete and trustworthy user experience.
Could Have	FR-14, FR-23, FR-26, FR-27, FR-28	Host review responses — valuable for platform trust but not blocking for MVP.
Won't Have (v1)	Payment processing, real-time chat, map-based search	Out of scope for the current academic delivery.

3.3 Non-Functional System Requirements

3.3.1 Technical & Architectural Constraints

ID	Constraint	Detail
NFR- T01	Backend Framework	The entire backend must be developed using Spring Boot REST APIs. No other backend framework is permitted.
NFR- T02	API Protocol	All inter-service and external communication must occur via RESTful HTTP APIs using JSON as the data interchange format.
NFR- T03	Microservices & Spring Cloud	The system must be decomposed into at minimum four independent microservices, with Spring Cloud Gateway for routing and Eureka for service discovery.
NFR- T04	Aspect-Oriented Programming	AOP must be used for centralized logging (all API calls) and security auditing (all role-sensitive operations).
NFR- T05	Containerization	The entire system (all microservices and databases) must be Dockerized and operable via docker-compose.
NFR-T06	Customer Service Module	The CS Service must be a standalone Spring Boot microservice with its own database; it must not share data storage with any other service.

3.3.2 Security

ID	Requirement	Detail
NFR- S01	Authentication	All API endpoints (except registration and login) must require a valid JWT in the Authorization header.
NFR- S02	Authorization	Role-Based Access Control must be enforced; a Guest must not be able to invoke Host or Admin endpoints, and vice versa.
NFR- S03	Password Storage	User passwords must be hashed using BCrypt before persisting to the database; plain-text passwords must never be stored.
NFR- S04	Token Expiry	Issued JWTs must have a configurable expiration time (default: 24 hours); expired tokens must be rejected.

3.3.3 Data & Persistence

ID	Requirement	Detail
NFR- D01	Database Isolation	Each microservice must maintain its own dedicated database to ensure data isolation and independent scaling.
NFR- D02	Persistence Technology	A relational database (MySQL) must be used as the persistent store for each service.
NFR- D03	Data Integrity	The Booking Service must enforce transactional consistency to prevent double-booking under concurrent request scenarios.

3.3.4 Performance

ID	Requirement	Detail
NFR- P01	Response Time	API responses for search and listing queries must complete within 3 seconds under normal load.
NFR- P02	Concurrent Users	The system should support at least 50 concurrent users without noticeable degradation.

3.3.5 Reliability & Availability

ID	Requirement	Detail
NFR- R01	Fault Isolation	If a single microservice becomes unavailable, remaining services must continue to operate independently.
NFR- R02	Circuit Breaker	Spring Cloud circuit breakers should be applied to inter-service calls to prevent cascading failures.

3.3.6 Maintainability

ID	Requirement	Detail
NFR- M01	Code Modularity	Each microservice must be independently buildable and deployable without modifying other services.
NFR- M02	Centralized Logging	AOP-based logging must produce structured logs (JSON format) consumable by monitoring tools.

4.0 Design & Implementation Constraints

- The backend must exclusively use Spring Boot; mixing other frameworks (e.g., Node.js, Django) is not permitted.
- All inter-service communication must be REST-based; message brokers (e.g., Kafka) are optional enhancements.
- **The Spring Cloud Gateway** must serve as the single entry point for all external client requests.
- **Eureka Service Discovery** must be used for dynamic service registration and lookup.
- Each microservice must have its own database container defined in the docker- compose configuration.
- JWT-based authentication must be validated at the Gateway level before forwarding requests to downstream services.
- AOP aspects must be implemented as separate `@Aspect` classes and must not pollute business logic layers.
- All Docker images must be based on official OpenJDK base images.

5.0 System Evolution

The following enhancements are anticipated in future versions of the NextStay? platform:

- **Payment Integration:** A dedicated Payment Microservice to process transactions via Stripe or PayPal.
- **Real-time Messaging:** An in-platform chat service enabling direct communication between Guests and Hosts.
- **Map Integration:** Geolocation-based property search using a mapping API (e.g., Google Maps).
- **Mobile Application:** Native iOS and Android clients consuming the existing REST APIs.
- **Machine Learning Recommendations:** A recommendation engine suggesting properties based on user history and preferences.
- **Multi-currency Support:** Dynamic pricing display based on the guest's preferred currency.
- **Live Agent Chat:** Upgrade the Customer Service ticketing system to support real-time chat between users and support agents using WebSockets.

6.0 Requirements Discovery Approaches

The following techniques were used to elicit and validate the requirements for NextStay?:

6.1 Competitive Analysis

The Airbnb platform (airbnb.com) was analyzed to identify essential features of a peer-to-peer accommodation marketplace. Features were deliberately scoped to meet academic project constraints while retaining the core rental lifecycle.

6.2 Scenario-Based Analysis

User scenarios (e.g., "a traveler searching for a property in Hurghada for three nights") were constructed for each role to identify functional requirements and edge cases such as double-booking and cancelled stays.

6.3 Project Guideline Review

The official SE-2 project guidelines and TA-issued requirements were used as normative constraints, ensuring that architectural decisions (microservices, AOP, Docker, Spring Cloud) are reflected as non-functional requirements.

6.4 Prototype Discussion

Team discussions around high-level wireframes of key screens (property search, booking form, host dashboard) were used to validate requirement completeness and uncover implicit expectations.

7.0 Requirements Validation Techniques

7.1 Requirements Review

All requirements were peer-reviewed by team members against the project guidelines to check for completeness, consistency, and correctness before inclusion in this document.

7.2 Traceability Matrix

Each system requirement is traceable to at least one user requirement (URS) and at least one system test condition to ensure full coverage.

7.3 Prototyping

High-level UI mockups of key flows (Guest booking flow, Host listing creation) will be constructed and validated against the functional requirements to confirm the requirements are implementable.

7.4 MoSCoW Prioritization

The MoSCoW scheme applied in Section 3.2 was reviewed by the team to ensure that Must Have requirements cover the graded deliverables and that no critical features were inadvertently deprioritized.

8.0 System Design and Models

The following diagrams constitute the required documentation artifacts as specified in the project guidelines. All diagrams are to be developed using UML notation. Diagram image placeholders are indicated below — each will be embedded in the final submission.

8.1 Use-Case Diagram

The Use Case Diagram details the interactions of the four primary actors — Guest, Host, Support Agent, and Admin — with the NextStay? system.

Actors:

- **Guest:** register, login, view listing, search properties, create booking, cancel booking, submit review, submit support ticket, track ticket status.
- **Host:** register, login, create listing, manage availability calendar, approve/decline booking, respond to review, submit support ticket, track ticket status.
- **Admin:** login, verify host, approve/flag listing, manage user accounts.
- **Support Agent:** login, view ticket dashboard, respond to ticket, resolve/close ticket.

Key Use Cases:

Use Case ID	Use Case Name	Primary Actor	Included / Extended
UC-01	Register Account	Guest / Host	
UC-02	{Abstract} Login	Guest / Host / Employees	Users' Admin, Employees' Admin and Support Agents
UC-03	Search Properties	Guest	
UC-04	Create Booking	Guest	
UC-05	Cancel Booking	Guest	
UC-06	Submit Review	Guest	Precondition: Booking status = Completed
UC-07	Create Listing	Host	<<include>> Verify Host Account
UC-08	Manage Calendar	Host	
UC-09	Approve/Decline Booking	Host	
UC-10	Respond to Review	Host	
UC-11	Verify Host	Admin	
UC-12	Approve/Flag Listing	Users' Admin	
UC-13	Manage User Accounts	Users' Admin	
UC-14	View Listing Detail	Guest	
UC-15	Delete Review	Guest	
UC-16	Submit Support Ticket	Guest / Host	
UC-17	Track Ticket Status	Guest / Host	
UC-18	Manage Ticket Dashboard	Support Agent	<<include>> Respond to Ticket
UC-19	Resolve / Close Ticket	Support Agent	
UC-20	Manage Support Agent Accounts	Employees' Admin	
UC-21	Assign Ticket to Agent	Employees' Admin	
UC-22	View Employee Performance	Employees' Admin	
UC-23	Flag Ticket as Action Needed with subtype selection	Support Agent	
UC-24	Resolve Action-Needed Ticket	Users' Admin	
UC-25	Initiate Refund	Users' Admin	

UC-26	Flag Review as Inappropriate	Support Agent	
-------	------------------------------	---------------	--

[Use Case Diagram — to be inserted as image]

8.1.1 Detailed Use-Cases Description

UC-01 – Register Account

Field	Description
ID Number – Name	UC-01 – Register Account
Short-Description / Goal	To create a new user profile with a specific role (Guest or Host).
Preconditions	The user has the NextStay? application open and is not logged in.
Postconditions	A new user record is persisted in the Identity Service database; a default profile is created.
Error situation	Registration fails; no data is committed to the database; system displays validation error.
Actor(s) / Initiator(s)	Guest / Host.
Trigger	User clicks on the "Sign Up" button.
Standard Process	1. User enters name, email, phone number, and desired role. 2. Identity Service validates the uniqueness of the email. 3. System hashes the password for security. 4. User record is saved to the database. 5. System generates a unique QR code for the profile.
Alternative Processes	Email already exists: System prompts the user to log in instead or use a different email.

UC-02 – {abstract} Login

Field	Description
ID Number – Name	UC-02 – Login {abstract}
Short-Description / Goal	To authenticate a user and provide a JWT for role-based API access.
Preconditions	The user has a registered account in the Identity Service database.
Postconditions	User is authenticated; a secure JWT is issued; user is redirected to their dashboard.

Error situation	Authentication fails; access is denied; no token is issued.
Actor(s) / Initiator(s)	Guest / Host / Support Agent / Admin.
Trigger	User attempts to sign in to access protected modules.
Standard Process	1. User enters credentials (email/phone and password). 2. Identity Service validates credentials against stored hashes. 3. System generates a JWT containing the user's role (Guest, Host, Support Agent, or Admin). 4. System returns the token to the client for subsequent REST API calls.
Alternative Processes	Incorrect Credentials: System displays an "Invalid credentials" message and logs the attempt via AOP.

UC-03 – Search Properties

Field	Description
ID Number – Name	UC-03 – Search Properties
Short-Description / Goal	To allow users to find available accommodations based on filters.
Preconditions	The Listing Service is active and reachable via the API Gateway.
Postconditions	A filtered list of available properties is displayed to the user.
Error situation	Token Expired: System returns 403 Forbidden. Service Down: System returns "Service Temporarily Unavailable."
Actor(s) / Initiator(s)	Guest.
Trigger	User enters search criteria (e.g., location, date, price) and clicks search.
Standard Process	1. Guest enters search parameters. 2. System filters properties in the Listing Service database. 3. Check Availability: System verifies status for requested dates. 4. Resulting list is returned to the Guest interface.

Alternative Processes	Token Expiration: If the JWT has expired, the system returns a 403 error and triggers a redirect to UC-02 (Login). Service Interruption: If the Listing Service is unreachable via the Gateway, the system returns a "Service Temporarily Unavailable" message.
------------------------------	--

UC-04 – Create Booking

Field	Description
ID Number – Name	UC-04 – Create Booking
Short-Description / Goal	To facilitate the reservation of a specific property for selected dates.
Preconditions	Guest is authenticated; property is available for the selected range.
Postconditions	A pending reservation is created in the Booking Service; dates are marked as reserved.
Error situation	Token Expired: 403 Forbidden error returned. Comm. Failure: System state remains unchanged.
Actor(s) / Initiator(s)	Guest.
Trigger	Guest clicks "Book Now" on a property listing page.
Standard Process	- Guest selects check-in and check-out dates. Prevent Double-Booking: Booking Service checks real-time availability via Listing Service. - System applies OCL constraints (check-out > check-in). - A "Pending" booking record is created in the database.
Alternative Processes	Token Expiration: Redirect to UC-02 (Login) upon 403 Forbidden error. Service Interruption: If the Booking Service cannot reach the Listing Service, return "Service Temporarily Unavailable."

UC-05 – Cancel Booking

Field	Description
ID Number – Name	UC-05 – Cancel Booking

Short-Description / Goal	To allow a guest to void a reservation and release the property dates.
Preconditions	An active booking exists associated with the Guest's account.
Postconditions	Booking status is updated to "Cancelled"; property dates are released.
Error situation	Token Expired: 403 Forbidden. Service Interruption: Status remains "Confirmed."
Actor(s) / Initiator(s)	Guest.
Trigger	Guest selects "Cancel" from their booking history.
Standard Process	1. Guest selects a specific active booking. 2. Booking Service validates if cancellation is within allowed timeframe. 3. Status is updated in the database. 4. Host is notified via the Notification Service.
Alternative Processes	Token Expiration: 403 error triggers redirect to UC-02 (Login) . Service Interruption: If the Booking Service cannot notify the Listing Service to release dates, return "Service Temporarily Unavailable."

UC-06 – Submit Review

Field	Description
ID Number – Name	UC-06 – Submit Review
Short-Description / Goal	To allow guests to provide feedback on a completed stay.
Preconditions	Guest is authenticated; Booking status = Completed.
Postconditions	Review is saved to the Review Service; property average rating is updated.
Error situation	Token Expired: 403 Forbidden. Service Down: Review not saved.
Actor(s) / Initiator(s)	Guest.
Trigger	Guest selects "Write a Review" for a finished stay.

Standard Process	1. Guest provides a star rating and written comment. 2. Review Service validates the stay status. 3. Review is persisted in the database.
Alternative Processes	Token Expiration: Redirect to UC-02 (Login) . Service Interruption: Return "Service Temporarily Unavailable" if the Review Service cannot reach the Booking Service for status verification.

UC-07 – Create Listing

Field	Description
ID Number – Name	UC-07 – Create Listing
Short-Description / Goal	To allow a Host to publish a new accommodation on the platform.
Preconditions	User is authenticated as a "Host".
Postconditions	New property listing is added to the database and visible to Guests.
Error situation	Token Expired: 403 Forbidden. Validation Failure: Listing invisible.
Actor(s) / Initiator(s)	Host.
Trigger	Host selects "Create New Listing" in the Host Dashboard.
Standard Process	1. Host enters property details and pricing. 2. <<include>> Verify Host Account: System ensures host has a "Verified" status. 3. Host uploads images via REST API. 4. Listing is saved in the Listing Service database.
Alternative Processes	Token Expiration: Redirect to UC-02 (Login) . Service Interruption: Return "Service Temporarily Unavailable" if the Listing Service cannot reach the Identity Service for verification.

UC-08 – Manage Calendar

Field	Description
ID Number – Name	UC-08 – Manage Calendar

Short-Description / Goal	To allow Hosts to manually block dates or adjust seasonal pricing.
Preconditions	Host is authenticated and has active listings.
Postconditions	Availability calendar is updated for the selected property.
Error situation	Token Expired: 403 Forbidden. Update Failure: Old availability persists.
Actor(s) / Initiator(s)	Host.
Trigger	Host accesses the "Calendar" section for a specific property.
Standard Process	1. Host selects specific dates on the UI calendar. 2. Host chooses an action (e.g., "Block Dates"). 3. Listing Service updates the availability table in the database. 4. Changes are reflected in search results immediately.
Alternative Processes	Token Expiration: Redirect to UC-02 (Login) . Service Interruption: Return "Service Temporarily Unavailable" if the Listing Service is unreachable.

UC-09 – Approve/Decline Booking

Field	Description
ID Number – Name	UC-09 – Approve/Decline Booking
Short-Description / Goal	To allow Hosts to respond to incoming reservation requests.
Preconditions	A "Pending" booking request exists for the Host's property.
Postconditions	Booking status is updated to "Confirmed" or "Declined."
Error situation	Token Expired: 403 Forbidden. Update Failure: Booking remains "Pending."
Actor(s) / Initiator(s)	Host.
Trigger	Host views a pending request in their notification center.

Standard Process	1. Host reviews Guest profile and requested dates. 2. Host selects "Approve" or "Decline." 3. Booking Service updates the record. 4. Guest is notified of the decision.
Alternative Processes	Token Expiration: Redirect to UC-02 (Login) . Service Interruption: Return "Service Temporarily Unavailable" if the Booking Service cannot reach the Notification Service.

UC-10 – Respond to Review

Field	Description
ID Number – Name	UC-10 – Respond to Review
Short-Description / Goal	To allow Hosts to reply to Guest feedback on their properties.
Preconditions	A review exists for a property owned by the Host.
Postconditions	The Host's response is published alongside the Guest's review.
Error situation	Token Expired: 403 Forbidden. Save Failure: Review remains without reply.
Actor(s) / Initiator(s)	Host.
Trigger	Host views a new review and selects "Reply."
Standard Process	1. Host enters a written response. 2. Review Service links the reply to the original Review ID. 3. System persists the response in the database.
Alternative Processes	Token Expiration: Redirect to UC-02 (Login) . Service Interruption: Return "Service Temporarily Unavailable" if the Review Service is unreachable.

UC-11 – Approve/Flag Listing

Field	Description
ID Number – Name	UC-12 – Approve/Flag Listing
Short-Description / Goal	To moderate platform content and ensure listing quality.
Preconditions	User is authenticated as an "Users' Admin."
Postconditions	Listing is either approved or flagged for correction.

Error situation	Token Expired: 403 Forbidden. Status Failure: Listing remains unchanged.
Actor(s) / Initiator(s)	Users' Admin.
Trigger	Users' Admin reviews a reported listing or a new submission.
Standard Process	1. Users' Admin checks listing for violations. 2. Users' Admin selects "Approve" or "Flag." 3. Listing Service updates the property status. 4. Host is notified if their listing was flagged.
Alternative Processes	Token Expiration: Redirect to UC-02 (Login) . Service Interruption: Return "Service Temporarily Unavailable" if the Listing Service is unreachable via the Gateway.

UC-12 – Manage User Accounts

Field	Description
ID Number – Name	UC-13 – Manage User Accounts
Short-Description / Goal	To provide Users' Admins control over all platform users for security.
Preconditions	User is authenticated as an "Users' Admin".
Postconditions	User accounts are modified or suspended.
Error situation	Token Expired: 403 Forbidden. Update Failure: Status fails to update.
Actor(s) / Initiator(s)	Users' Admin.
Trigger	Users' Admin accesses the User Management dashboard.
Standard Process	1. Users' Admin searches for a specific User ID. 2. Users' Admin modifies account status (e.g., Suspended). 3. Identity Service updates the record in the database. 4. AOP: System logs the Users' Admin action for auditing.
Alternative Processes	Token Expiration: Redirect to UC-02 (Login) . Service Interruption: Return "Service Temporarily Unavailable" if the Identity Service is unreachable.

UC-13 – View Listing Detail

Field	Detail
ID Number – Name	UC-14 – View Listing Detail
Short-Description / Goal	A Guest views the full details of a property listing including photos, amenities, location, pricing, host profile, and existing guest reviews.
Preconditions	Listing must exist with status Active. Guest is authenticated or browsing as a visitor.
Postconditions	Guest has viewed listing details. No system state is changed.
Error situation	If listing is not found or not Active: system displays "Listing not available" message.
Actor(s) / Initiator(s)	Guest.
Trigger	Guest clicks on a listing from search results.
Standard Process	1. Guest selects a listing from search results. 2. System retrieves listing details from Listing Service. 3. System retrieves aggregated reviews from Review Service. 4. System displays full listing page to Guest.
Alternative Processes	Service Interruption: If the Listing Service is unreachable, return "Service Temporarily Unavailable."

UC-14 – Submit Support Ticket

Trigger	User clicks "Submit a Support Request" in the Support section.
Standard Process	1. User navigates to Support section. 2. User fills in subject and enters description. 3. User submits the form. 4. System creates a ticket with status Open and returns a ticket ID. 5. User receives confirmation notification. Support Agents are notified.
Alternative Processes	Token Expiration: Redirect to UC-02 (Login).

Field	Detail
ID Number – Name	UC-15 – Submit Support Ticket
Short-Description / Goal	An authenticated user submits a support request by providing a subject and description of their issue.
Preconditions	User must be authenticated as Guest or Host.
Postconditions	Ticket exists with status Open and is visible to Support Agents. User receives confirmation.
Error situation	If required fields are missing system prompts user to complete the form. Token Expired: 403 Forbidden.
Actor(s) / Initiator(s)	Guest / Host.

UC-15 – Track Ticket Status

Field	Detail
ID Number – Name	UC-16 – Track Ticket Status
Short-Description / Goal	An authenticated user views the history and status of their submitted support tickets.
Preconditions	User must be authenticated; at least one ticket must exist for the user.
Postconditions	No state change. User is informed of their ticket status and conversation thread.
Error situation	If no tickets exist: system displays "No support tickets found." Token Expired: 403 Forbidden.
Actor(s) / Initiator(s)	Guest / Host.
Trigger	User navigates to "My Tickets" from the Support section.
Standard Process	1. User navigates to My Tickets. 2. System retrieves all tickets belonging to the user. 3. User sees ticket list with status, subject, category, and last updated. 4. User selects a ticket to view the full conversation thread.
Alternative Processes	Token Expiration: Redirect to UC-02 (Login). Service Interruption: Return "Service Temporarily Unavailable" if CS Servi
	Validation Failure: Return field-level error messages and keep form open.

ce is
unreach

--	--

UC-16 – Manage Tickets Dashboard (Employees’ Admin)

Field	Detail
ID Number – Name	UC-17
Short-Description / Goal	Manage Ticket Dashboard
Preconditions	Employees’ Admin
Postconditions	An Employees’ Admin views all platform support tickets, filters by status and assigns tickets to Support Agents
Error situation	User must be authenticated with Employees’ admin role.
Actor(s) / Initiator(s)	1. Employees’ Admin navigates to the tickets Dashboard. 2. System retrieves all tickets with optional filter. 3. Employees’ Admin selects a ticket. 4. Employees’ Admin assigns ticket to a Support Agent.
Trigger	If no tickets match the filter: system displays "No tickets found."
Standard Process	Ticket status updated; user notified.
Alternative Processes	<<include>> Respond to Ticket

UC-17 – In Progress Tickets (Support Agent)

Field	Detail
ID Number – Name	UC-17
Short-Description / Goal	Support Agent starts resolving the ticket
Preconditions	Support Agent
Postconditions	An Employees’ Admin views all platform support tickets, filters by status and assigns tickets to Support Agents
Error situation	User must be authenticated with Support Agent role.
Actor(s) / Initiator(s)	1. Support Agent navigates to the tickets Dashboard. 2. System retrieves all assigned tickets with optional filter. 3. Support Agent selects a ticket. 4. Support Agent marks it as “In Progress” and starts resolving issue 5. Support Agent marks ticket as resolved after issue is resolved
Trigger	If no tickets match the filter: system displays "No tickets found."

Standard Process	Ticket status updated; user notified.
Alternative Processes	<<include>> Respond to Ticket

UC-18 – Resolve / Close Ticket

Field	Detail
ID Number – Name	UC-18 – Resolve / Close Ticket
Short-Description / Goal	A Support Agent marks a ticket as Resolved once the user's issue has been addressed, and to allow the ticket to transition to Closed after a cooldown period.
Preconditions	Ticket must exist with status In Progress. Support Agent must be authenticated.
Postconditions	Ticket status is updated to Resolved; user is notified. <u>After 7 days with no further activity, ticket auto-transitions to Closed.</u>
Error situation	Token Expired: 403 Forbidden. Invalid Transition: If ticket is not In Progress, system rejects the Resolved action and returns a 409 error.
Actor(s) / Initiator(s)	Support Agent.
Trigger	Support Agent determines the issue has been fully addressed and clicks "Mark as Resolved."
Standard Process	- Support Agent reviews the ticket thread and confirms issue is resolved. - Agent clicks "Mark as Resolved."
	- CS Service updates ticket status to Resolved. - User is notified that their ticket has been resolved. - System starts a 7-day cooldown timer. - If no further user activity after 7 days: status auto-transitions to Closed.
Alternative Processes	User Reopens: If the user replies after the ticket is Resolved (within cooldown), status reverts to In Progress and Agent is notified. Token Expiration: Redirect to UC-02 (Login).

8.2 Activity Diagrams

8.3 Class Diagram

The Class Diagram defines the domain model for the NextStay? system, showing all entities, their attributes, operations, and relationships across the four microservices.

Primary domain classes:

- User (id, name, email, passwordHash, role: Enum{GUEST, HOST, ADMIN}, isVerified, isFlagged, isActive, createdAt)
- Agent (id, name, email, passwordHash, role: Enum{SUPPORT_AGENT, SUPPORT_LEAD, ADMIN}, isActive, lastLoginAt, createdAt)
- Listing (id, hostId, title, description, location, amenities, pricePerNight, maxGuests, status: Enum{INACTIVE, ACTIVE, SUSPENDED, REJECTED}, averageRating)
- AvailabilitySlot (id, listingId, date, isBlocked)
- Reservation (id, guestId, listingId, checkInDate, checkOutDate, numGuests, status: Enum{PENDING, CONFIRMED, COMPLETED, CANCELLED, REJECTED}, createdAt)
- Review (id, reservationId, guestId, listingId, rating: int[1..5], comment, createdAt)
- HostReviewResponse (id, reviewId, hostId, responseText, createdAt)
- SupportTicket (id, userId, userRole: Enum{GUEST, HOST}, assignedAgentId, subject, description, status: Enum{OPEN, IN_PROGRESS, RESOLVED, CLOSED}, createdAt, updatedAt)
- TicketMessage (id, ticketId, senderId, senderRole: Enum{USER, SUPPORT_AGENT}, messageText, sentAt)

OCL Constraints (mandatory for Class Diagram):

- context Reservation inv ValidDates: self.checkInDate < self.checkOutDate
- context Reservation inv NoDoubleBooking: Reservation.allInstances()->forall(r | r <> self implies not (r.listingId = self.listingId and r.status <> ReservationStatus::CANCELLED and no (r.checkOutDate <= self.checkInDate or r.checkInDate >= self.checkOutDate)))
- context Review inv ValidRating: self.rating >= 1 and self.rating <= 5
- context Review inv ReviewAfterCompletion: self.reservation.status = ReservationStatus::COMPLETED
- context Listing inv ValidPrice: self.pricePerNight > 0
- context User inv UniqueEmail: User.allInstances()->isUnique(u | u.email)
- context SupportTicket inv ValidStatus: Set{'OPEN','IN_PROGRESS','RESOLVED','CLOSED'}->includes(self.status)
- context SupportTicket inv ClosedAfterResolved: self.status = 'CLOSED' implies self.status@pre = 'RESOLVED'
- context Reservation inv RejectedFromPending: self.status = ReservationStatus::REJECTED implies self.status@pre = ReservationStatus::PENDING

8.4 Sequence Diagrams

The Sequence Diagrams show the RESTful communication between microservices during the key booking flow, illustrating how the Spring Cloud Gateway, Identity Service, Listing Service, and Booking Service collaborate.

8.5 Entity Relationship Diagram (ERD)

Each microservice maintains its own isolated database. The ERD for each service is described below.

Identity Service DB

Table	Key Columns
users	id (PK), name, email (UNIQUE), password_hash, role, is_verified, is_flagged, is_active, created_at
agents	id (PK), name, email (UNIQUE), password_hash, role (SUPPORT_AGENT / SUPPORT_LEAD / ADMIN), is_active, last_login_at, created_at

Listing Service DB

Table	Key Columns
listings	id (PK), host_id (FK → users.id), title, description, location, price_per_night, max_guests, status, average_rating
amenities	id (PK), listing_id (FK), name
availability_slots	id (PK), listing_id (FK), slot_date, is_blocked

Booking Service DB

Table	Key Columns
reservations	id (PK), guest_id, listing_id, check_in_date, check_out_date, num_guests, status, created_at

Review Service DB

Table	Key Columns
reviews	id (PK), reservation_id (UNIQUE), guest_id, listing_id, rating (1-5), comment, created_at
host_responses	id (PK), review_id (FK UNIQUE), host_id, response_text, created_at

Customer Service DB

Table	Key Columns
support_tickets	id (PK), user_id, user_role, subject, description, category, status, created_at, updated_at

ticket_messages	id (PK), ticket_id (FK → support_tickets.id), sender_id, sender_role, message_text, sent_at
-----------------	--

[ERD Diagram — to be inserted as image]

9.0 System Architecture Overview

NextStay? is built on a microservices architecture orchestrated via Spring Cloud. The following describes the high-level architecture.

9.1 Service Decomposition

Service	Port	Responsibility	Database
Eureka Discovery Server	8761	Service registry for dynamic service lookup.	None
API Gateway (Spring Cloud)	8080	Single entry point; JWT validation; request routing.	None
Identity & Access Service	8081	Registration, login, JWT issuance, RBAC, profile management.	MySQL: identity_db
Listing & Catalog Service	8082	Property CRUD, search, availability calendar, listing verification.	MySQL: listing_db
Booking & Reservation Service	8083	Reservation lifecycle, double-booking prevention, scheduler.	MySQL: booking_db
Review & Feedback Service	8084	Review submission, host responses, rating aggregation.	MySQL: review_db
Customer Service	8085	Support ticket creation, ticket lifecycle management, agent responses, ticket history.	MySQL: cs_db

9.2 Cross-Cutting Concerns (AOP)

The following concerns are implemented via **Spring AOP @Aspect classes** and are applied across all services:

- Request Logging Aspect: Logs method name, input parameters, execution duration, and response status for every API call.
- Security Audit Aspect: Logs all attempts to access role-restricted endpoints, including the requesting user ID, role, endpoint, and outcome (allowed/denied).
- Exception Handling Aspect: Intercepts unhandled exceptions and logs a structured error event before propagating to the global exception handler.

9.3 Docker Compose Structure

The docker-compose.yml file defines the following containers:

- eureka-server
- api-gateway
- identity-service + identity-db (MySQL)
- listing-service + listing-db (MySQL)
- booking-service + booking-db (MySQL)
- review-service + review-db (MySQL)
- customer-service + cs-db (MySQL)

Services register with Eureka on startup; the Gateway discovers service instances via Eureka for dynamic routing.

9.4 System Architecture

The NextStay platform utilizes a **Cloud-Native Microservices Architecture**. The backend is decoupled into five independent Spring Boot business domains (Identity, Listing, Booking, Review, Support), each maintaining its own isolated MySQL database to ensure data sovereignty.

External client requests are routed through a centralized **Spring Cloud API Gateway**, which handles cross-cutting concerns such as JWT validation and routing. Internal service discovery and load balancing are orchestrated by **Netflix Eureka**. Communication between microservices (e.g., the Booking Service requesting data from the Listing Service) is handled synchronously via declarative REST bindings using **OpenFeign**.

9.5 List of User Interfaces

The frontend is a React-based Single Page Application (SPA) divided into distinct role-based experiences:

- **Guest Interfaces:**
 - **Explore/Search View:** For filtering and discovering active properties.
 - **Property Detail View:** Displays images, amenities, pricing, and reviews.
 - **My Reservations:** Tracks Pending, Confirmed, Cancelled, and Completed bookings.
- **Host Interfaces:**
 - **Host Dashboard:** Displays aggregate statistics, earnings, and recent alerts.
 - **My Listings / Edit Listing:** Forms to create, update, and manage property details and availability.
 - **Booking Requests:** A management view to Approve or Decline pending reservations.
- **Admin & Support Interfaces:**
 - **Ticket Dashboard:** A Kanban or list view for Support Agents to track Open, In Progress, and Resolved user issues.
 - **Verification Dashboard:** Used by Users' Admins to approve, flag, or suspend new property listings.

9.6 Design Patterns

The system implementation actively utilizes several recognized software design patterns to ensure clean code and maintainability:

1. **The Builder Pattern (Creational):** Extensively utilized via Lombok (`@Builder`) within Entity and Data Transfer Object (DTO) classes (e.g., `ReservationResponse`). This pattern allows for the safe and readable construction of complex objects without needing massive constructors or multiple setter calls.
2. **The Repository Pattern (Structural):** Implemented via Spring Data JPA interfaces (e.g., `ListingRepository`, `ReservationRepository`). This pattern abstracts the database layer, allowing the business logic services to query data using domain terminology (`findByGuestId`) rather than writing raw SQL queries.

9.7 The Strategy used to implement the use cases

NextStay was implemented using an **API-First, Frontend-Backend Separation Strategy**.

1. **Stateless Security:** Use cases requiring user authentication rely on stateless JSON Web Tokens (JWT). The React frontend stores the token and role, appending them to every Axios request.
2. **Role-Based Controller Endpoints:** Each use case is mapped to a specific REST controller endpoint protected by role-checking logic. For example, UC-09 Approve/Decline Booking is implemented by exposing `PUT /{id}/approve` and strictly verifying the `X-User-Role` header is `HOST` before passing the request to the service layer.
3. **Cross-Cutting Isolation:** Rather than hardcoding logging and audit trails into every use case method, Aspect-Oriented Programming (AOP) was used to intercept method calls globally, keeping the business logic focused purely on the use case requirements.